udp FACULTAD DE INGENIERÍA Y CIENCIAS

“Exploring Multi-Armed Bandit Problem in Reinforcement Learning: A Comparative Study of Epsilon-Greedy, UCB, and Thompson Sampling Algorithms”

PROJECT PYTHON FOR ENGINEERS

Name student: María Paz Silva

Professor: Thomas Maedler

Date: December 17, 2023

Abstract

Reinforcement learning (RL) is a rapidly evolving field with applications in various domains of engineering and machine learning. One of the fundamental problems in reinforcement learning is the multi-armed bandit (MAB) problem, which models the exploration versus exploitation dilemma.

This project aims to delve deeper into the problem of discrete reward multi-armed bandits, focusing on three algorithms: Epsilon-Greedy, Upper Confidence Bound (UCB) and Thompson Sampling, understand their underlying principles and compare their performance to maximize expected rewards.

We will implement these algorithms using Python and tools to graph and visualize their performance. The performance of the algorithms will be evaluated based on cumulative reward and cumulative regret metrics.

The results indicate that the Thompson Sampling and UCB algorithms outperform the Epsilon-Greedy algorithm in terms of sampling performance. This suggests that Thompson Sampling and UCB are more adept at striking a balance between exploration and exploitation, leading to superior decision making in uncertain environments.

Table of contents

Table of illustrations 4
Introduction 5
Methods 6
E-greedy Algorithm 7
Upper Confidence Bound (UCB) Algorithm 8
Thompson Sampling Algorithm 9
Metrics 10
Statistical metrics 10
Python methods 13
Summary 14
Bibliography 16

Table of illustrations

Illustration 1 Reinforcement learning feedback framework 6
Illustration 2 Exploration - Exploitation 7
Illustration 3 e-greedy Algorithm 8
Illustration 4 Upper Confidence Bound Algorithm 9
Illustration 5 Thompson Sampling Algorithm 10
Illustration 6 Comparison of avg. rewards from e-greedy, Thompson sampling and UCB . 14
Illustration 7 Comparison of cumulative reward and cumulative regret 15

Introduction

Reinforcement learning (RL) has become a fundamental tool in engineering and machine learning, with applications ranging from autonomous vehicles to recommendation systems. RL involves an agent learning to make decisions by taking actions in an environment to maximize cumulative rewards. Policies, which guide the agent's actions based on the current state, play a crucial role in this process. The goal is to find the optimal policy that maximizes the expected cumulative reward over time. This process is iterative and adaptive, improving over time as the agent gains more experience and knowledge about the environment. A fundamental problem that RL addresses is the multiple armed bandit (MAB) problem, which encapsulates the exploration-exploitation dilemma. This challenge focuses on balancing known rewards with exploring new options for potentially greater rewards. This trade-off is critical in decision-making under uncertainty, which is prevalent in stock trading, clinical trials, and A/B testing. **The MAB problem is a simplified version of the full RL problem, making it an ideal starting point for understanding the complexities of decision-making in uncertain environments.

The MAB problem manifests itself in a variety of ways, including continuous and discrete reward scenarios. Continuous rewards involve outcomes modeled as continuous random variables, requiring sophisticated exploration-exploitation strategies. In contrast, discrete rewards simplify decision making by treating rewards as distinct values, as discussed by Sutton and Barto (2018). These different scenarios require different approaches and algorithms, highlighting the versatility and adaptability of RL. Stationary MAB problems maintain constant reward distributions, creating a stable environment for decision making. The upper confidence bound (UCB) algorithm, explored by Auer et al. (2002), takes advantage of this stability. On the other hand, non-stationary MAB problems introduce dynamic changes, which requires adaptability. Kocsis and Szepesvári (2006) propose the Exp3 algorithm to address these dynamic environments. These algorithms are designed to handle the inherent uncertainty and variability in the reward distributions, making them robust to changes in the environment. Context-free bandit problems are based solely on historical results, while contextual bandit problems integrate external contextual information. Langford and Zhang (2007) explore context-free bandit scenarios, emphasizing simplicity and efficiency. Li et al. (2010) delves into the contextual problems of bandits, showing the impact of contextual information on decision making. The integration of contextual information allows the agent to make more informed decisions, potentially leading to higher cumulative rewards.

This project focuses on implementing and comparing three fundamental MAB algorithms in Python: Epsilon-Greedy, UCB, and Thompson Sampling. Each algorithm balances exploration and exploitation uniquely, with different advantages and disadvantages. Epsilon-Greedy, although simple, may lack efficiency, while UCB and Thompson Sampling, being more complex, offer potential performance benefits. These algorithms represent different approaches to the exploration-exploitation trade-off, providing a comprehensive overview of the strategies used in RL.

The main objective of the project is to deepen the understanding of these algorithms, their mathematical principles, and their performance to maximize the expected rewards. The Python programming language is used for the implementation. The performance of the algorithm is evaluated by optimizing parameters and obtaining cumulative reward and regret

metrics, which provides insights into its effectiveness in solving the MAB problem. The project also aims to provide a practical, hands-on experience with RL, complementing the theoretical understanding with practical application. This report clarifies the mathematical underpinnings of the algorithms and metrics used for performance comparison. The Python code, described in detail, emphasizes key insights gained during execution. It ends with a summary of the project objectives, the results obtained and possible future improvements to the code. The report serves as a comprehensive guide to the project, providing both a theoretical background and a practical guide to the implementation and evaluation of the algorithms.

Methods

In the realm of reinforcement learning, a pivotal concept is the decision-making process under conditions of uncertainty. This concept is eloquently encapsulated by Sutton and Barto in their seminal work, “Reinforcement Learning: An Introduction.” They describe an agent’s objective as maximizing its cumulative reward over time. However, the challenge lies in the fact that the exact rewards for each action are unknown. Instead, they can only be estimated through the agent’s experiences. This uncertainty in decision-making is exemplified by the multi-armed bandit (MAB) problem. As articulated by Lai and Robbins in their 1985 paper, the MAB problem presents an agent with multiple slot machines, or ‘arms,’ each offering a different and unknown reward distribution. The agent’s task is to strategically choose which arm to pull at each time step, navigating through the uncertainty to maximize its total reward.

The environment, the agent and the reward constitute the triad of components of the MAB problem, which reflect the set of states and actions, the decision maker, and the feedback, respectively. According to Auer et al. (2002), "the problem of choice under conditions of uncertainty is a fundamental challenge" and its resolution requires a delicate balance between exploration and exploitation.

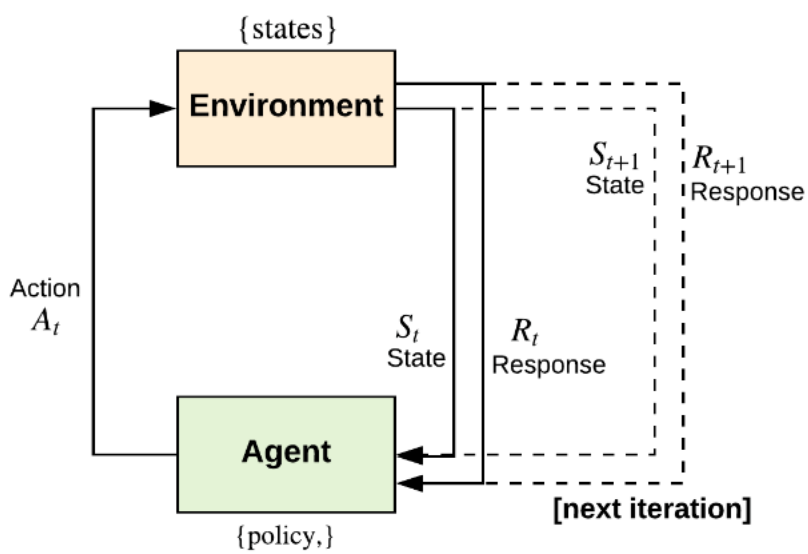

Illustration 1 Reinforcement learning feedback framework

The balance between exploration and exploitation is a central theme, well articulated in the empirical evaluation of Thompson Sampling by Chapelle and Li (2011), where exploration involves testing different weapons to gather information about their reward distributions,

while exploitation involves Test different weapons to gather information on the distribution of their rewards by pulling on the arm that currently appears to offer the highest expected reward.

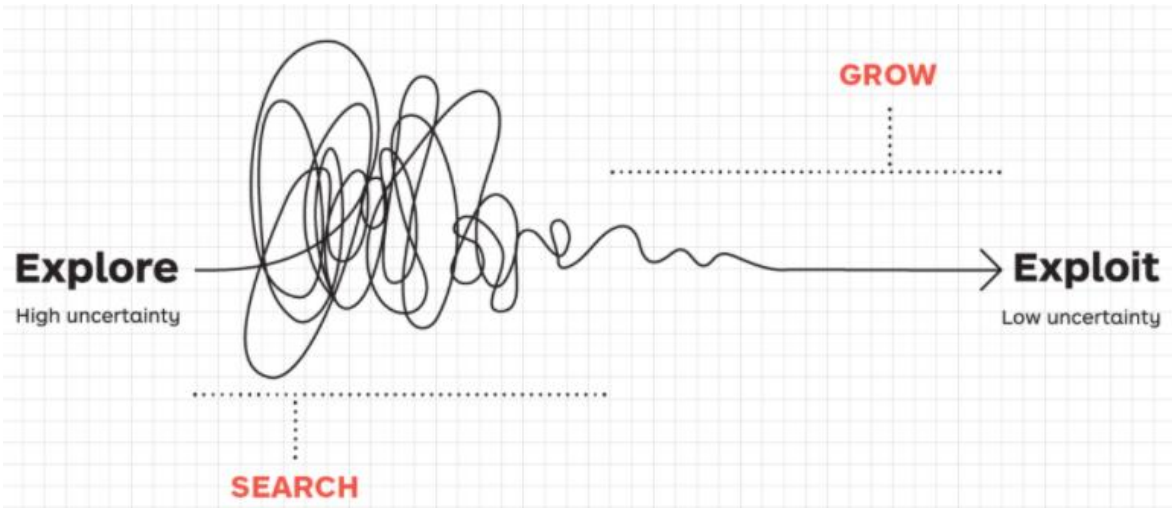

Illustration 2 Exploration - Exploitation

Several algorithms have been developed, as succinctly stated by Auer et al., to address the MAB problem. These include the widely known epsilon-greedy, UCB (Upper Confidence Bound) and Thompson Sampling.

E-greedy Algorithm

The e-greedy algorithm navigates the exploration-exploitation trade-off by introducing a parameter ε. At each step, it exploits the best option with a probability of $1-\varepsilon$ or explores a random option with a probability of ε. The parameter ε determines the degree of exploration, and its tuning influences the algorithm's behavior. A smaller ε leans towards exploitation, while a larger ε emphasizes exploration. This strategic approach ensures a harmonious equilibrium between experimenting with new options and adhering to the known optimal choice. The update mechanism involves adjusting ε based on observed rewards, facilitating continuous adaptation.

The policy used by e-greedy involves selecting the option with the highest estimated value during exploitation and a random option during exploration. This dual-action strategy allows the algorithm to strike a balance between exploiting known valuable options and exploring potentially rewarding alternatives.

The mathematical representation of the e-greedy policy is:

$$\pi(a|s) = \begin{cases} 1-\varepsilon+\dfrac{\varepsilon}{|A(s)|}, & \text{if } a = \operatorname{argmax}_{a \in A(s)} Q(s,a) \\ \dfrac{\varepsilon}{|A(s)|}, & \text{otherwise} \end{cases}$$

Where $\pi(a|s)$ is the probability of choosing action a given state s, $A(s)$ is the set of all possible actions in state s and $Q(s,a)$ represents the value of taking action a in state s.

The update rule for e-greedy is:

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{n}(r_n - Q_n(a))$$

Where Q_n represents the value of taking action a in state s and r_n is reward received at time step n.

Algorithm 1 $\epsilon - greedy$

Initialize $Q(b) = 0, N(b) = 0,\ b = 1, 2, ..., k$

1: **for** $t = 1, 2, ..., N$ **do**

2: $\quad B(t) \leftarrow \begin{cases} \text{argmax}_b\, Q_t(b) & \text{w.t } 1-\epsilon \\ \text{randomly chosen from } b & \text{w.t } \epsilon \end{cases}$

3: $\quad N_t(B(t)) \leftarrow N_t(B(t)) + 1$

4: $\quad Q_t(B(t)) \leftarrow Q_{t-1}(B(t)) + \frac{1}{N_{t-1}(B(t))}(R_{t-1}(B(t)) - Q_{t-1}(B(t)))$

5: **end for**

Illustration 3 e-greedy Algorithm

Upper Confidence Bound (UCB) Algorithm

The UCB algorithm introduces a confidence parameter, often denoted as c, to facilitate a more informed exploration strategy. It selects an option with the highest upper confidence bound, calculated as the sum of the average reward and c times the uncertainty associated with that average. The parameter c influences the level of exploration, with higher c values promoting more exploration. This strategic approach favors the selection of options with high uncertainty, encouraging the algorithm to explore less-explored choices.

The policy employed by UCB involves selecting the option with the maximum upper confidence bound. This decision-making process ensures that options with high uncertainty, and thus potentially high rewards, are prioritized. The algorithm's continuous learning and adaptation occur through dynamic updates based on observed rewards.

The mathematical representation of the UCB policy is:

$$a_t = \arg\max_{a \,\epsilon\, A(s)} \left[Q(s,a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Where $Q(s,a)$ is the estimated reward of action a, $N_t(a)$ is the number of times action a has been selected up to time t, and c is a parameter controlling the degree of exploration.

The update rule for the UCB algorithm is the same as the e-greedy algorithm.

Algorithm 2 Upper Confidence Bound

```
for t = 1, 2, ..., k do
    B(t) ← t
    N_t(B(t)) ← 1
    Q_t(B(t)) ← R(B(t))
end for

for t = k + 1, ..., N do
    B(t) ← argmax_b(Q_t(b) + c√(log(t)/N_t(b)))
    N_t(B(t)) ← N_t(B(t)) + 1
    Q_t(B(t)) ← Q_{t-1}(B(t)) + 1/N_{t-1}(B(t)) (R_{t-1}(B(t)) − Q_{t-1}(B(t)))
end for
```

Illustration 4 Upper Confidence Bound Algorithm

Thompson Sampling Algorithm

The Thompson Sampling algorithm incorporates a Bayesian perspective by maintaining a probability distribution over the expected reward for each option. The parameter *prior* the prior distribution, capturing the algorithm's initial beliefs about the options. The decision-making process involves sampling from these distributions to select options. This Bayesian approach inherently balances exploration and exploitation, as options with high uncertainty in their distributions are more likely to be explored.

The policy employed by Thompson Sampling entails selecting options based on the sampled values from the probability distributions. This approach allows the algorithm to naturally balance exploration and exploitation. Similar to other algorithms, Thompson Sampling continuously updates its probability distributions based on observed rewards, ensuring adaptation to the evolving environment.

The policy of Thompson Sampling is to select the action with the highest sampled reward. The sampling rule is:

$$a_t = \text{argmax}_{a \in A(s)}[r_t(a)]$$

Where $r_t(a)$ is a reward sampled from the posterior distribution of action a at time step t.

The update rule for the Thompson Sampling algorithm is:

$$\text{Beta}(S_a + 1, F_a + 1)$$

Where S_a is the number of successes (rewards of 1) and F_a is the number of failures (rewards of 0) for action a.

Algorithm 3 Tompson Sampling

Initialize Beta parameter $(\alpha_b, \beta_b),\ b = 1, 2, ..., k$

```
for t = 1, 2, ..., N do

    for b = 1, 2, ..., k do
      sample Q_t(b) ~ Beta(α_b, β_b)
    end for

    B(t) ← argmax_b(Q_t(b))
    α_B(t) ← α_B(t) + R(B(t))
    β_B(t) ← β_B(t) − R(B(t)) + 1

end for
```

Illustration 5 Thompson Sampling Algorithm

Metrics

These metrics and graphs are essential tools for understanding and comparing the performance of different algorithms on the MAB problem. They provide quantitative measures of performance and visual aids to interpret the results.

Average reward at each time step: This metric represents the average reward obtained by the algorithm at each time step. The graph of average reward over time provides information about the learning progress of the algorithm. A rising curve indicates that the algorithm is effectively learning to make better decisions over time.

Average rewards metric over time steps: This metric is a way of measuring the cumulative reward that an agent receives from taking actions in an environment. It is calculated by dividing the total reward by the number of time steps.

Average Explored trial count: This metric shows the average number of times each option has been tested (or "armed") at each time step. It helps to understand the balance between exploration and exploitation that the algorithm makes. A balanced weapon count indicates a focus on exploration, while a high count of a few weapons indicates exploitation.

Cumulative reward: Cumulative reward is the total reward that the algorithm has accumulated up to a given time step. A graph of cumulative reward over time provides a visual representation of the overall performance of the algorithm. A steeper slope indicates a higher rate of reward accumulation, which is desirable.

Cumulative regret: Cumulative regret is the difference between the total reward that would have been obtained by always choosing the best option and the total reward obtained by the algorithm. A flatter curve indicates lower regret and therefore better performance.

Statistical metrics

The statistical methods used to compare the average reward are the one-way ANOVA test and the Tukey HSD test. These methods are suitable for this purpose because they can test

the significance of multiple variables in a data set and compare differences between means of values rather than pairs of values.

The one-way ANOVA test is based on the F distribution, which compares the variation between groups with the variation within groups. The null hypothesis of this test is that all group means are equal, and the alternative hypothesis is that at least one group mean is different from the rest.

$$F = \frac{\sigma_1^2}{\sigma_2^2}$$

Where σ_1^2 is variance of the first sample and σ_2^2 is variance of the second sample.

The Tukey HSD test is a post hoc test that follows a one-way ANOVA to discover which specific groups have different means from each other. It uses a Studentized rank distribution to compare all possible pairs of means and adjust for multiple comparisons. The null hypothesis of this test is that there are no significant differences between any pair of group means, and the alternative hypothesis is that there are significant differences between at least one pair of group means. The test statistic employed by Tukey's HSD is q, which is calculated for a comparison between means A and B as:

$$q_s = \frac{\bar{Y}_A - \bar{Y}_B}{SE_{\bar{Y}_A - \bar{Y}_B}}$$

choose A and B such that the difference between their means is positive.

$$SE_{\bar{Y}_A - \bar{Y}_B} = \sqrt{\frac{MS_{within}}{2}\left(\frac{1}{n_B} + \frac{1}{n_A}\right)}$$

can obtain the value for MSwithin from your ANOVA table.

Code description

To initiate, they present the pseudo code that encapsulates the functioning of the original code.

1. Import necessary libraries
2. Define MultiArmedBandit class
3. Define EpsilonGreedy, ThompsonSampling, and UCB subclasses
4. Initialize parameters and environment
5. For each strategy:
    a. Optimize parameters
    b. Run strategy with optimized parameters
    c. Plot performance
6. Perform statistical comparison of strategies

The code begins by importing necessary libraries such as ‘numpy’ for numerical operations, ‘matplotlib.pyplot’ for data visualization, and ‘scipy’ and ‘statsmodels’ for statistical analysis.

The key methods in the code are:

‘run_episode’: This method simulates an episode of the multi-armed bandit problem. It uses the estimated reward probabilities to decide which arm to pull and updates these probabilities based on the observed rewards.

‘update’: This method updates the estimated reward probabilities based on the observed rewards. Each strategy updates these probabilities differently.

‘plot_algorithm_performance’: This method visualizes the performance of the algorithm. It plots the average reward over time steps, the average reward across all time steps, and the average count of explored arms.

The ‘MultiArmedBandit’ class is the foundation for the multi-armed bandit problem. It includes methods to execute an episode ‘run_episode’, update the estimated reward probabilities ‘update’, and visualize the algorithm's performance ‘plot_algorithm_performance’.

The ‘EpsilonGreedy’, ‘ThompsonSampling’, and ‘UCB’ classes are subclasses of ‘MultiArmedBandit’. Each implements a specific strategy for the multi-armed bandit problem by overriding the ‘run_episode’ method of the superclass.

The code includes a parameter optimization process for each strategy. It tests different parameter values and identifies the optimal ones. For the Epsilon-Greedy strategy, it tests various epsilon values. For Thompson Sampling, it tests different alpha and beta values. For UCB, it tests different values of c. After determining the best parameters, the code executes each strategy again with these parameters. It then visualizes the performance using the ‘plot_algorithm_performance’ method of each class.

Finally, the code performs a one-way ANOVA test and a Tukey's HSD test to statistically compare the performance of the different strategies.

The main enhancements in this code compared to a standard implementation are the inclusion of parameter optimization, performance visualization, and statistical comparison. In the parameter optimization process, the code uses the optimized parameters to run the algorithm, unlike a standard implementation that might display how the performance of each algorithm changes with each parameter value. The performance visualization in this code includes plots of the average reward over time steps, the average reward across all time steps, and the average count of explored arms for each algorithm using the optimized parameter value. This is more comprehensive than the basic performance graphs obtained in a standard implementation. This code also includes the calculation and plotting of cumulative reward and regrets metrics, providing a holistic view of the algorithm's performance over time. Lastly, the statistical comparison feature, which is not found in a standard implementation, performs a one-way ANOVA test and a Tukey's HSD test after running all the strategies to statistically compare their performance. This provides a rigorous basis for comparing the effectiveness of the strategies.

Python methods

In the execution of the provided code, a deep understanding of several Python methods and programming concepts was gained. The code uses classes to encapsulate related data and functions. For instance, the 'MultiArmedBandit' class was defined with attributes like 'num_arm' and 'reward_probs', and methods like 'run_episode' and 'update'. The 'EpsilonGreedy', 'ThompsonSampling', and 'UCB' classes are subclasses of the 'MultiArmedBandit' class, demonstrating the use of inheritance in Python.

The project also involved defining and using functions. For instance, the 'run_episode' method was defined in each class to simulate an episode of the multi-armed bandit problem. This method uses the estimated reward probabilities to decide which arm to pull and updates these probabilities based on the observed rewards.

A unique feature of Python that was learned is multiple assignment, which allows assigning values to multiple variables at once. This is evident in the line 'exp_reward_prob, explored_cnt = self.update(arm, reward, exp_reward_prob, explored_cnt)', where the 'update' method returns two values, which are assigned to 'exp_reward_prob' and 'explored_cnt', respectively.

The 'np.random.choice' function was used to generate a random sample from a given 1-D array. In the line 'reward = np.random.choice([0, 1], p=[1-self.reward_probs[arm], self.reward_probs[arm]])', it's used to randomly select a reward (0 or 1) based on the reward probabilities of the selected arm.

The functions 'np.where' and 'np.argmax' were used to find the indices of elements in an array that satisfy a condition. In the line 'greedy_arm_indices = np.where(exp_reward_prob == exp_reward_prob.max())[0]', 'np.where' is used to find the indices of the arms with the highest expected reward probability.

List comprehension, a concise way to create lists, was used in the line 'succ_fail = [list(x) for x in self.prior]' to create a deep copy of 'self.prior'. Each element of 'self.prior' is converted to a list and added to 'succ_fail'.

The 'np.random.beta' function was used to draw samples from a Beta distribution. In the line 'samples = [np.random.beta(s + 1, f + 1) for s, f in succ_fail]', it's used to draw a sample for each arm based on the success ('s') and failure ('f') counts.

Lastly, the underscore '_' was used to ignore values that are not needed. In the lines 'avg_rewards_ts, _, _, _ = ts.run_iteration(num_ite, num_trial)' and similar ones, '_' is used to ignore the values returned by 'run_iteration' that are not needed.

Summary

This project further explores the multi-armed bandit (MAB) problem, a fundamental challenge in reinforcement learning. The MAB problem summarizes the classic exploration versus exploitation dilemma. The goal of the project was to implement and evaluate the performance of three fundamental algorithms that address the MAB problem with discrete rewards: Epsilon-Greedy, Upper Confidence Bound (UCB), and Thompson Sampling.

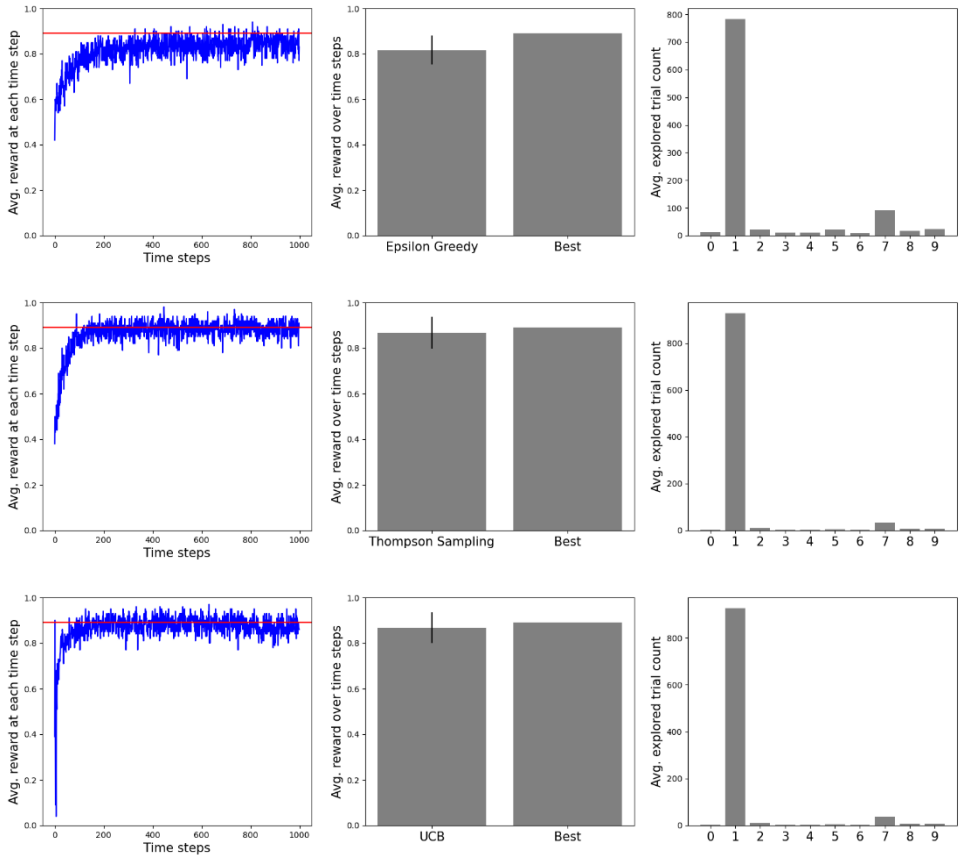

Illustration 6 Comparison of avg. rewards from e-greedy, Thompson sampling and UCB

The results indicate that the Thompson Sampling and UCB algorithms outperform the Epsilon-Greedy algorithm in terms of sampling performance. This suggests that Thompson Sampling and UCB are more adept at striking a balance between exploration and exploitation, leading to superior decision making in uncertain environments.

The key difference causing the superior performance of Thompson Sampling and UCB is their ability to exploit uncertainty in the reward distribution for more effective exploration. By considering the uncertainty around the average reward, they can explore more intelligently and avoid getting stuck in suboptimal actions, which can happen with Epsilon-Greedy due to its random exploration strategy.

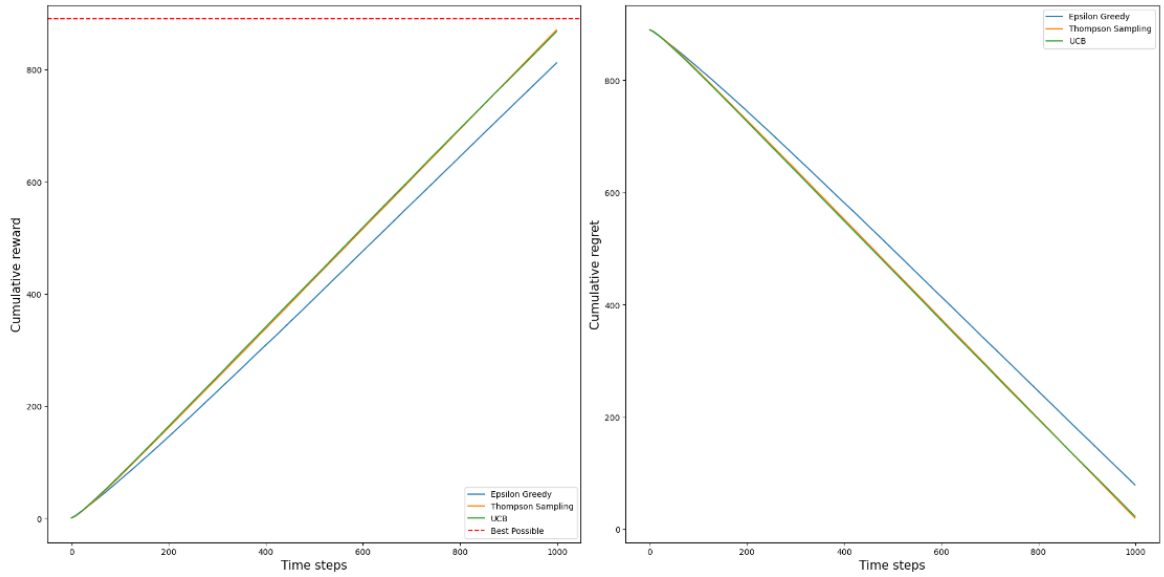

Illustration 7 Comparison of cumulative reward and cumulative regret

Statistical analysis, including one-way ANOVA and Tukey HSD test, provides strong evidence of these differences. Statistical analyses, including a one-way ANOVA test and Tukey HSD test, demonstrate significant differences in average rewards between Epsilon-Greedy and Thompson Sampling and UCB. There is no significant difference between the average rewards of Thompson Sampling and UCB.

The project also identifies future improvements the code and expand the scope of the project. One such extension involves modeling scenarios in which the reward changes over time. This would require adapting the algorithms to handle non-stationary reward distributions, a common scenario in many real-world applications that the current project does not address. Another possible extension is to model situations in which the reward is continuous rather than discrete. This would lead to the contextual bandit problem, a more complex variant of the MAB problem where the reward depends on the context. In this scenario, algorithms would need to consider contextual information when making decisions.

In conclusion, this project represents a crucial step towards understanding and addressing the MAB problem in reinforcement learning. The knowledge acquired in this project lays the foundation for future research aimed at developing more effective and intelligent decision-making systems.

Bibliography

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine learning, 47*(2-3), 235-256.

Bubeck, S., & Cesa-Bianchi, N. (2012). Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends® in Machine Learning, 5*(1), 1-122.

Chapelle, O., & Li, L. (2011). An empirical evaluation of Thompson sampling. *In Advances in neural information processing systems*, 2249-2257.

Chu, W., Li, L., Reyzin, L., & Schapire, R. (2011). Contextual bandits with linear payoff functions. *In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* , 208-214.

Kocsis, L., & Szepesvári, C. (2006). Discounted ucb. *In 2nd PASCAL Challenges Workshop, 2*, 51-134.

Lai, T. L., & Robbins, H. (1985). Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics, 6*(1), 4-22.

Langford, J., & Zhang, T. (2007). The epoch-greedy algorithm for multi-armed bandits with side information. *Advances in neural information processing systems*, 20.

Minai, Y. (2023). *Exploring Multi-Armed Bandit Problem: Epsilon-Greedy, Epsilon-Decreasing, UCB, and Thompson Sampling*. Obtenido de https://github.com/yuumii-san/RL_tutorial_notebook/blob/main/multi_armed_bandit/multi_armed_bandit.ipynb

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction. *MIT press*. Obtenido de http://incompleteideas.net/sutton/book/the-book-2nd.html